LangGraph: Graph-Based Orchestration for Stateful LLM Applications

and its Applications

CAQ413 Seminar

Seminar Report submitted in partial fulfillment for the degree
B.Tech in Computer Science and Engineering (Artificial Intelligence)

Submitted by

Nikhil Jimmy Thomas (Reg No: MUT23CA055)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
(Affiliated to APJ Abdul Kalam Technological University)

October 2026

Muthoot Institute of Technology and Science (Autonomous)
Kochi

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

CERTIFICATE

This is to certify that the seminar report entitled “LangGraph: Graph-Based Orchestration for Stateful LLM Applications and its Applications” submitted by Nikhil Jimmy Thomas (Reg No: MUT23CA055) of Semester VII B.Tech Computer Science and Engineering (AI) is a bonafide record of the work done under my supervision during the academic year 2026–2027.

Guide	**Coordinators**	**Head of the Department**
Ms. Nimmi M K	Ms. Thasni K N & Ms. Vishnupriya Vijay	Dr. Cerene Mariam Abraham
Asst. Professor	Asst. Professors	Associate Professor
Dept. of AI & DS	Dept. of AI & DS	Dept. of AI & DS

ACKNOWLEDGEMENTS

This report would not exist without the help of a number of people, and I want to thank them here.

I thank Dr. Neelakantan P. C., Principal, Muthoot Institute of Technology & Science, for the facilities and the academic setting in which this seminar was prepared.

I am grateful to Dr. Cerene Mariam Abraham, Head of the Department of Artificial Intelligence and Data Science, for her support and for giving me the chance to take up this topic.

My seminar guide, Asst. Prof. *Ms. Nimmi M K*, deserves special thanks. She read my early drafts, asked what the base paper really showed, and kept me from wandering off topic. Much of the shape of this report comes from those conversations.

I also thank my seminar coordinators, Asst. Prof. *Ms. Thasni K N* and Asst. Prof. *Ms. Vishnupriya Vijay*, who reviewed the work at each stage and told me plainly what had to be fixed.

My thanks go as well to the management of the college and to the teaching and non-teaching staff of the department for their help during the semester. Last, I thank my classmates in S7 AI&DS, whose questions during the practice runs improved the presentation, and my family, who put up with the late nights.

Nikhil Jimmy Thomas
(Reg. No: MUT23CA055)

Abstract

You can ask a large language model (LLM) a question and get a decent answer. Ask it to do a real job and it usually needs several steps. It has to decide what to do next, try again when something breaks, and remember what it did earlier. A fixed chain of prompts cannot loop back. A fully autonomous agent can, but it is hard to predict and harder to debug. LangGraph, an open-source library from LangChain, takes a middle path. The developer draws the application as a graph. Nodes are steps, edges are the moves between them, and every node shares one state. The model thinks inside the nodes, and the graph decides the order.

This seminar looks at LangGraph through a paper by Duan and Wang, *Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI* (arXiv:2411.18241). They let LangGraph control the flow and CrewAI organise agents into role-based teams, and they show the idea on an email assistant, a loop that writes and reviews code, and a system that checks and forwards work orders. The report covers LangGraph's core ideas, six related papers, the paper's architecture and workflow, and its strengths, weaknesses and uses. One thing stood out to me. The paper's results are case studies, not measurements, so its claims about efficiency are still untested.

Keywords: LangGraph, CrewAI, Large Language Models, Multi-Agent Systems, Agent Orchestration.

Contents

List of Figures **iii**

List of Tables **iv**

List of Abbreviations **v**

1 Introduction **vi**

2 Literature Survey **viii**
2.1 ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2023) . . . viii
2.2 Reflexion: Language Agents with Verbal Reinforcement Learning (Shinn et al., 2023) . . . viii
2.3 AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation (Wu et al., 2023) . . . ix
2.4 MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework (Hong et al., 2024) . . . ix
2.5 A Survey on Large Language Model based Autonomous Agents (Wang et al., 2023) . . . ix
2.6 Agent AI with LangGraph: A Modular Framework for Enhancing Machine Translation Using LLMs (Wang and Duan, 2024) . . . ix

3 Materials and Methods **xi**
3.1 LangGraph: Core Concepts . . . xi
3.2 CrewAI Framework . . . xii
3.3 Development Environment . . . xiii
3.4 Evaluation Criteria . . . xiii

4 Methodology **xiv**
4.1 Design Rationale . . . xiv
4.2 System Architecture . . . xiv
4.3 Integrating LangGraph with CrewAI . . . xv
4.4 Workflow . . . xv
4.5 Worked Example: Automatic Email Handling . . . xvi

5 Results and Performance **xxi**
5.1 Application Case 1: Code Generation with Code Review . . . xxi
5.2 Application Case 2: Automatic Work-Order Processing . . . xxii
5.3 Key Findings . . . xxii

5.4 Comparison with Other Approaches xxiii
5.5 Discussion xxiii

6 Advantages **xxiv**

7 Disadvantages **xxv**

8 Applications **xxvi**

9 Conclusion **xxvii**

A Bibliography **xxviii**

B Presentation Slides **xxx**

C Sample Reference Papers **xxxvii**

D Viva Questions and Answers **xxxviii**

E Similarity Index Report **xl**

F Alignment with Sustainable Development Goals (SDG) **xli**

List of Figures

3.1 A minimal LangGraph agent: nodes, a conditional edge (router) and a loop . xii

4.1 Two-layer architecture: LangGraph controls the flow and CrewAI does the work inside each node xv
4.2 Workflow of the LangGraph+CrewAI system xvi
4.3 Email workflow graph built with LangGraph+CrewAI (redrawn from Duan and Wang [1], Fig. 1) xvii
4.4 LangGraph+CrewAI workflow code (transcribed from [1], Fig. 2) xviii
4.5 Agent definition in CrewAI (transcribed from [1], Fig. 3) xix
4.6 Task definition in CrewAI (transcribed from [1], Fig. 4) xx

5.1 Code generation and code check loop built with LangGraph+CrewAI (redrawn from [1], Fig. 5) xxi
5.2 Work-order processing with a crew of agents and a vector store (redrawn from the description in [1]; the agents' labels are illustrative) xxii

B.1 Seminar presentation slides (1–3) xxx
B.2 Seminar presentation slides (4–6) xxxi
B.3 Seminar presentation slides (7–9) xxxii
B.4 Seminar presentation slides (10–12) xxxiii
B.5 Seminar presentation slides (13–15) xxxiv
B.6 Seminar presentation slides (16–18) xxxv
B.7 Seminar presentation slides (19–20) xxxvi

C.1 Base paper: "Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI" (first page) [1] xxxvii

List of Tables

2.1 Summary of the literature survey x

3.1 Development environment used in the base paper xiii

5.1 Qualitative comparison of LLM application approaches xxiii

F.1 Mapping of the seminar topic to UN Sustainable Development Goals xli

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
FAISS	Facebook AI Similarity Search
HITL	Human-in-the-Loop
JS / TS	JavaScript / TypeScript
LLM	Large Language Model
MAS	Multi-Agent System
ML	Machine Learning
PM	Product Manager
RAG	Retrieval-Augmented Generation
SDG	Sustainable Development Goal
SOP	Standard Operating Procedure

Chapter 1

Introduction

Ask a large language model a question and you get an answer. Ask it to do a job, and things get harder. Real jobs come in steps.

Think about a support bot for a college website. A student types “my fee receipt is wrong”. The bot has to work out that this is a billing problem, not a login problem, and send it to the right place. It then pulls the student’s record, writes a reply and checks the reply before it goes out. A coding assistant has it worse. It writes some code, runs it, reads the error, and tries again. Sometimes that takes five rounds.

In other words, real work branches and loops, and the program has to remember what happened two steps back. Neither of the usual ways of building LLM apps copes well with that. A *chain* runs its steps in a fixed order, top to bottom. You always know what it will do next, which is nice, but it can never go back, and its steps keep no common memory. A *free-running agent* is the opposite: the model decides everything, even when to stop. You get flexibility. You also get something that is hard to test, where one wrong guess in the first minute can spoil the whole run.

LangGraph sits between these two. It is an open-source library from LangChain, released in 2024 under the MIT licence, with Python and JavaScript/TypeScript versions [2]. Its design borrows ideas from graph-processing systems such as Google’s Pregel [3]. You draw your application as a graph. Each node is a step. Each edge says where to go next. Every node reads from, and writes to, one shared state. The developer decides which paths exist, and the model does the thinking inside each step. It works with many model providers, and people use it for single agents, agent teams, retrieval pipelines and flows that wait for a human to say “yes”.

The base paper for this seminar is by Duan and Wang [1]. Their question is simple: what happens if you put LangGraph together with CrewAI [4], a framework for building teams of agents with roles? In their design, LangGraph runs the flow. It picks the next step, decides when to branch or loop, and keeps the state. CrewAI runs the team. It decides which agent does which task. The authors try this on three problems: an email assistant, a loop that writes and reviews code, and a ticket-handling system.

Objectives of the Seminar

- Explain why an LLM app with many steps needs explicit control, and why chains and free agents are not enough on their own.
- Describe the building blocks of LangGraph (state, nodes, edges, conditional edges, checkpoints).
- Review earlier work on LLM agents and agent teams.
- Study the design in the base paper, how its workflow runs, and the cases it was tried on.
- Judge where graph-based orchestration pays off and where it gets in the way, and list where it is used today.

Here is how the rest of the report is laid out. Related papers come first, in Chapter 2, then the tools in Chapter 3. Chapter 4 walks through the base paper's method; its results are in Chapter 5. After that come three short chapters, on advantages, on disadvantages and on applications (Chapters 6–8), before Chapter 9 closes the report.

Chapter 2

Literature Survey

I picked six papers for this survey. Each one changed how people build LLM agents, and together they explain why a tool like LangGraph was needed. For each paper I note what it does, what it showed, and where it falls short if you want a team of agents you can actually control.

2.1 ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2023)

ReAct [5] has the model take turns between three kinds of output. First a *thought*, where it reasons about what to do. Then an *action*, such as a Wikipedia search. Then an *observation*, which is whatever the tool sent back. Being able to look things up halfway through means less guessing. ReAct beat reasoning-only and acting-only baselines on HotpotQA, ALFWorld and WebShop, and most tool-using agents today follow the same pattern. The weak spot is the structure. The whole run is one straight loop with no saved state. If the model makes a mistake early, that mistake rides along to the end.

2.2 Reflexion: Language Agents with Verbal Reinforcement Learning (Shinn et al., 2023)

Reflexion [6] lets an agent learn from failure without retraining the model. When an attempt fails, the agent writes itself a short note, in plain English, about what went wrong. On the next try it reads the note first. That small trick gave large gains on coding and decision tasks. The catch is the success signal. Reflexion only works when the agent can tell clearly whether it succeeded, and many real tasks never give you that. Every retry also costs extra model calls.

2.3 AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation (Wu et al., 2023)

AutoGen [7] comes from Microsoft. You build an app out of several agents that solve the problem by chatting with each other. An agent can be a model, a tool, a person, or a mix. That makes AutoGen very flexible. But the order of work comes out of the conversation itself, so you cannot say in advance what the system will do. In practice agents can go round in circles or drift away from the task.

2.4 MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework (Hong et al., 2024)

MetaGPT [8] copies a software company. The authors write standard operating procedures (SOPs) into the prompts. Agents play a product manager, an architect and an engineer, and pass structured documents down an assembly line, checking each other's work as they go. It showed that structure cuts down the mess you get when agents just chat. Still, the pipeline is fixed and built mainly for software projects. Running that many agents also burns a lot of tokens.

2.5 A Survey on Large Language Model based Autonomous Agents (Wang et al., 2023)

Wang et al. [9] tried to map the whole area. They suggest that any LLM agent can be described with four parts: a *profile* (its role), *memory*, *planning* and *action*. They also review where agents are used and how they are tested. I found it the most useful paper for getting the big picture. Being a survey, though, it gives you no tool to build with, and new agent papers appear faster than any survey can keep up.

2.6 Agent AI with LangGraph: A Modular Framework for Enhancing Machine Translation Using LLMs (Wang and Duan, 2024)

This earlier paper by the same two authors [10] uses LangGraph as a router. It reads the input and hands it to a translation agent for English, French or Japanese, each built on a model such as GPT-4o. As an example of routing in LangGraph, it is neat and concrete.

The testing is thin, though. Only a few languages are covered, and the system depends on a paid model.

Summary

Read in order, the papers tell a story. ReAct taught agents to act. Reflexion taught them to learn from mistakes. AutoGen let them talk, and MetaGPT made them follow procedures. But the same problem keeps coming back. Either the flow is rigid and cannot branch or loop, or it is handed to the model and becomes hard to control. What was missing was a way for the developer to lay out the allowed paths, with shared state and room to retry, while the model still does the clever part of each step. That is the gap LangGraph, and the LangGraph+CrewAI pairing, set out to fill.

Table 2.1: Summary of the literature survey

Paper	**Key idea**	**Main limitation**
ReAct [5]	Alternates reasoning with tool calls	Single linear loop; no saved state; errors carry forward
Reflexion [6]	Agent writes notes on its failures and retries	Needs a clear success signal; extra model calls
AutoGen [7]	Agents solve tasks by conversing	Flow comes from the conversation, so it is hard to control
MetaGPT [8]	Role-based agents follow SOPs	Fixed pipeline aimed at software projects
Agent survey [9]	Profile, memory, planning, action framework	No concrete orchestration tool
LangGraph for MT [10]	LangGraph routes text to translation agents	Few languages; small evaluation

Chapter 3

Materials and Methods

Nothing in this work is trained, and no dataset is involved. So “materials”, for this chapter, simply means the software the system is put together from. I go through those pieces one by one and finish with the yardsticks the paper applies to its own results.

3.1 LangGraph: Core Concepts

LangGraph builds LLM apps as graphs that keep state between steps [2]. The base paper points to three things it adds over other LLM frameworks: loops, close control over the flow, and memory that survives from one step to the next [1]. It can run one agent or many, in a hierarchy or in sequence. Its pieces are listed below and drawn in Fig. 3.1.

- **State** is the shared data, usually a typed dictionary. Every node reads it and updates it. A reducer can say how an update is merged, for example adding a new message to a list instead of replacing the list.
- **Node** is just a Python function. It does one step, such as calling the model or a tool, and returns the part of the state it changed.
- **Edge** is a fixed move: after node A, run node B.
- **Conditional edge** is a small router function that looks at the state and picks the next node. Branches and loops are made from these.
- **START and END** mark where the graph begins and finishes.
- **Checkpointer** saves the state after every step. That means a run can be paused, inspected, picked up again after a crash, or held until a person approves it.

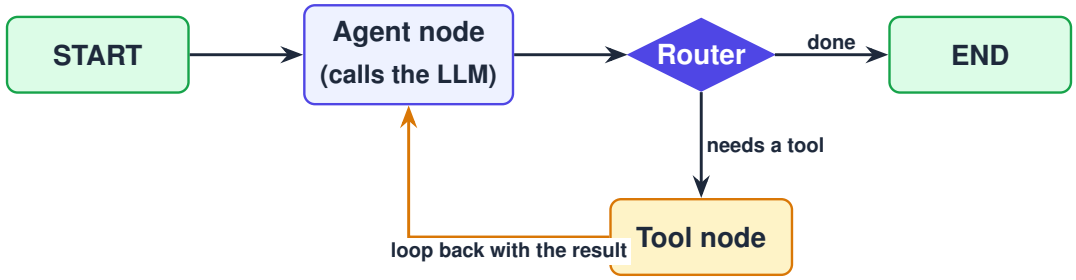


Figure 3.1: A minimal LangGraph agent: nodes, a conditional edge (router) and a loop

Building a LangGraph app takes five steps. Define the state. Add nodes. Connect them with edges. Compile. Run. The tiny example below answers one question with one model call.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):          # 1. define the state
    question: str
    answer: str

def answer(state: State):        # 2. a node is a plain function
    return {"answer": llm.invoke(state["question"])}

g = StateGraph(State)
g.add_node("answer", answer)
g.add_edge(START, "answer")      # 3. connect the edges
g.add_edge("answer", END)
app = g.compile()                # 4. compile
app.invoke({"question": "What is LangGraph?"})   # 5. run
```

3.2 CrewAI Framework

CrewAI [4] is an open-source framework for agents that play roles and work on a problem together. You give each agent a role, a goal and some tools. There are four pieces:

- An **agent** does the work. It decides things and can talk to the other agents. You define one by writing its role and goal, plus a short backstory.
- A **task** is one job. It comes with a description, names the agent responsible, and lists the tools that agent may use.
- A **tool** is anything an agent can use, such as reading email or searching documents.
- A **crew** is the team of agents plus the process that decides the order of their tasks.

The models behind CrewAI can be hosted ones, OpenAI's for instance, or local ones served through Ollama [11]. Roles are written by the developer. When one agent finishes, CrewAI passes the work on to the next without extra code [4]. It was built for jobs that take several steps and need a decision or two on the way.

3.3 Development Environment

Table 3.1 shows what the base paper used to build its cases [1].

Table 3.1: Development environment used in the base paper

Component	**Tool**	**Purpose**
Multi-agent framework	CrewAI [4]	Organises and coordinates the agents' work
Graph workflow	LangGraph [2]	Shares state between nodes and controls the flow
Workflow tracking	LangSmith [12]	Monitors and audits each run
Embedding model	Ollama embeddings [11]	Turns work-order text into vectors for retrieval
Vector database	FAISS [13]	Stores the vectors and finds similar ones quickly

3.4 Evaluation Criteria

The base paper is exploratory. It has no accuracy figures, and nothing on speed or cost either. What it does instead is judge LangGraph+CrewAI on four qualities, and since there is nothing better to go on, I borrow the same four for this report:

- **Control**: can the developer define and restrict the flow?
- **Information flow**: how easily do results pass between steps and agents?
- **Collaboration**: how well is the work split among agents with different roles?
- **Extensibility**: how easily can existing tools and agents, such as LangChain ones, be plugged in?

Chapter 4

Methodology

4.1 Design Rationale

The paper starts from a plain observation. One LLM agent cannot cope with a complex task on its own. A team of agents can, but only with two things in place: tight control of the flow, and good teamwork [1]. So the authors split the job. LangGraph's graph handles the control and carries information between agents. CrewAI's crews handle who does what. The paper then asks two questions. How should agents be designed on LangGraph so the flow is precisely controlled? And how can CrewAI make those agents good at more kinds of task?

4.2 System Architecture

There are two layers, and they meet at the shared state (Fig. 4.1).

- **LangGraph is the control layer.** The user's task enters the graph. Nodes stand for agent steps. Edges, some of them conditional, decide which step runs next, and the state carries data along.
- **CrewAI is the team layer.** Inside a node, a crew does the real work. Each agent has a role and a goal, tasks are handed out, and the agents combine their results into the node's output.

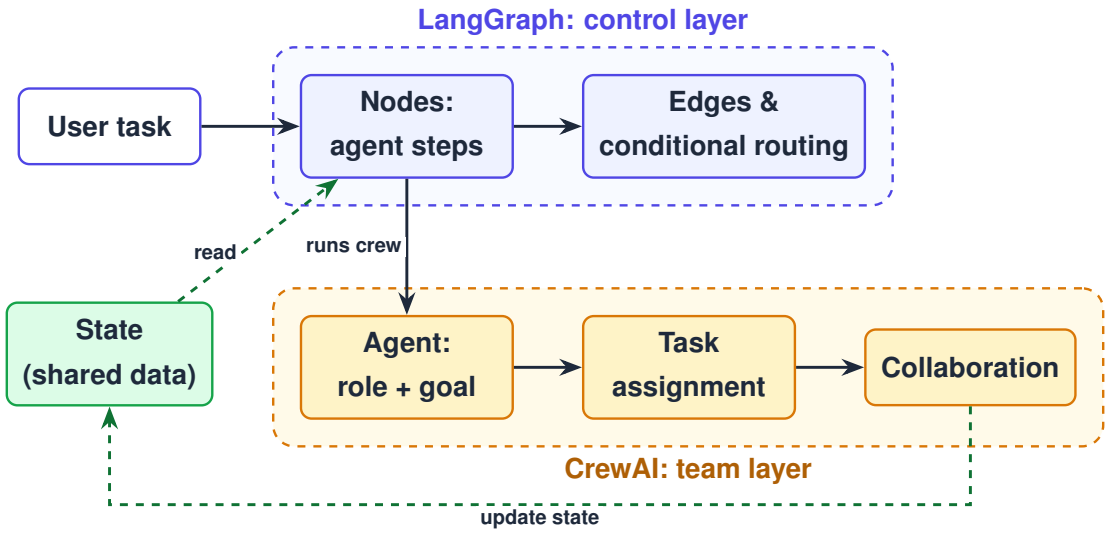


Figure 4.1: Two-layer architecture: LangGraph controls the flow and CrewAI does the work inside each node

4.3 Integrating LangGraph with CrewAI

Why combine them at all? The authors' answer is that a hard task becomes easier to manage once the graph handles the flow and a crew handles the people, so to speak. The combined system can also be wired to tools a team already uses. Because CrewAI gives each task one owner, it is always clear who did what. And since CrewAI talks to LangChain, agents someone has already written in LangChain can be reused as they are [1].

Put simply:

- **LangGraph owns the flow**: the order of steps, the branches, the loops and the state.
- **CrewAI owns the teamwork**: which agent takes which task, and how they cooperate.

4.4 Workflow

Fig. 4.2 follows one task through the system. LangGraph takes the task and sends it to a node. The crew in that node does the work and writes the result back to the state. Then a conditional edge asks one question: are we done? If not, the flow moves on to the next step. If yes, the graph ends.

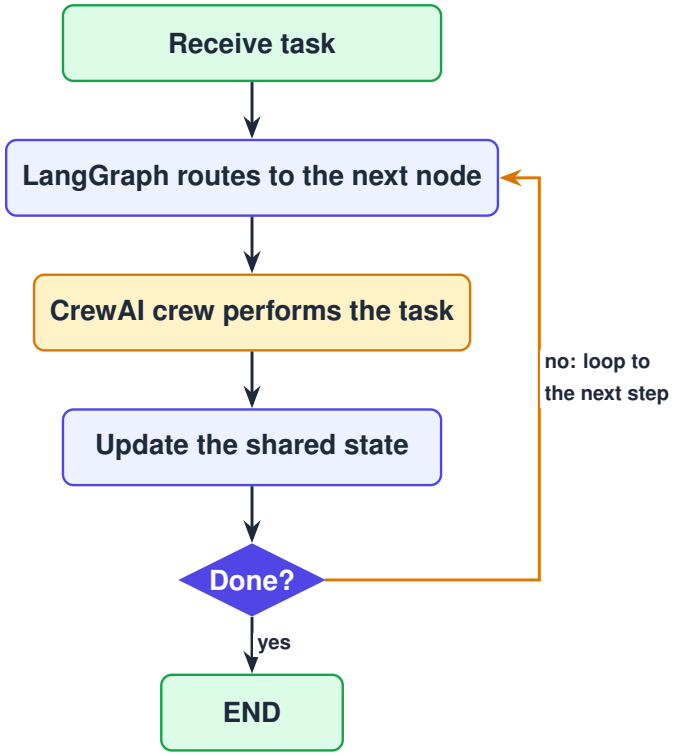


Figure 4.2: Workflow of the LangGraph+CrewAI system

4.5 Worked Example: Automatic Email Handling

The authors borrow the email example from CrewAI's documentation and rebuild it on LangGraph+CrewAI [1]. The job is cut into small steps that run on their own.

Fig. 4.3 shows the graph. It starts at `check_new_emails`. If new mail has arrived, a conditional edge sends it to `draft_responses`. If not, it goes straight to `wait_next_run`. After the wait, the graph goes back and checks the inbox again. That last loop is the interesting bit. It turns a script that runs once into an assistant that keeps going, and a plain chain simply cannot express it.

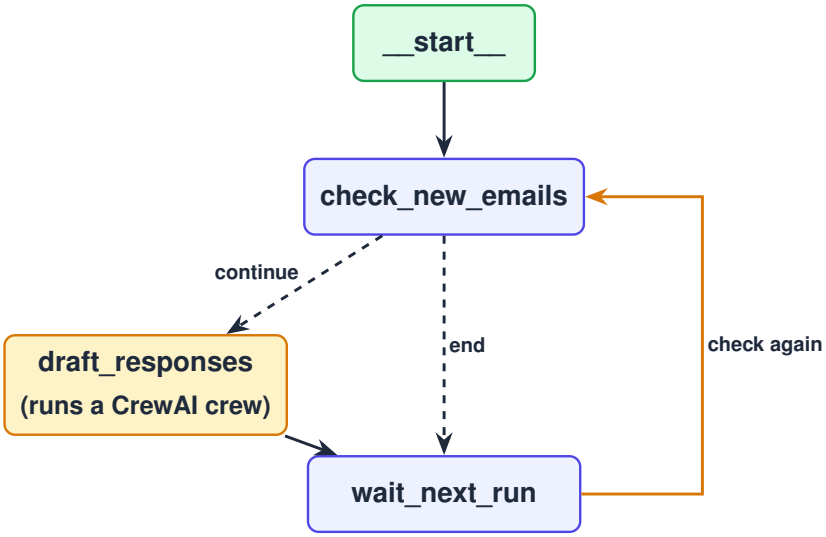


Figure 4.3: Email workflow graph built with LangGraph+CrewAI (redrawn from Duan and Wang [1], Fig. 1)

The code behind the graph (Fig. 4.4) follows the same five steps as Chapter 3. Look at the `draft_responses` node. Its function is `EmailFilterCrew().kickoff`. In other words, a whole CrewAI crew runs as a single LangGraph node. When I first read the code, this one line made the whole design click for me.

```python
from dotenv import load_dotenv
load_dotenv()
from langgraph.graph import StateGraph
from .state import EmailsState
from .nodes import Nodes
from .crew.crew import EmailFilterCrew

class WorkFlow():
    def __init__(self):
        nodes = Nodes()
        workflow = StateGraph(EmailsState)
        workflow.add_node("check_new_emails", nodes.check_email)
        workflow.add_node("wait_next_run", nodes.wait_next_run)
        workflow.add_node("draft_responses", EmailFilterCrew().kickoff)
        workflow.set_entry_point("check_new_emails")
        workflow.add_conditional_edges(
            "check_new_emails",
            nodes.new_emails,
            {"continue": 'draft_responses', "end": 'wait_next_run'})
        workflow.add_edge('draft_responses', 'wait_next_run')
        workflow.add_edge('wait_next_run', 'check_new_emails')
        self.app = workflow.compile()
```

Figure 4.4: LangGraph+CrewAI workflow code (transcribed from [1], Fig. 2)

Each agent in the crew is written up with a role and a goal, and with a backstory too (Fig. 4.5). In this case the “Senior Email Analyst” is the one who bins newsletters and promotions. The matching task (Fig. 4.6) spells out the job itself. It also fixes the shape of the answer and names the agent who owns it.

```
class EmailFilterAgents():
    def __init__(self):
        self.gmail = GmailToolkit()

    def email_filter_agent(self):
        return Agent(
            role='Senior Email Analyst',
            goal='Filter out non-essential emails like newsletters and promotional
    content',
            backstory=dedent("""\
                As a Senior Email Analyst, you have extensive experience in email
    content analysis.
                You are adept at distinguishing important emails from spam,
    newsletters, and other
                irrelevant content. Your expertise lies in identifying key patterns
    and markers that
                signify the importance of an email."""),
            verbose=True,
            allow_delegation=False
        )
```

Figure 4.5: Agent definition in CrewAI (transcribed from [1], Fig. 3)

```
class EmailFilterTasks:
    def filter_emails_task(self, agent, emails):
        return Task(
            description=dedent(f"""\
                Analyze a batch of emails and filter out
                non-essential ones such as newsletters, promotional content and
    notifications.

                Use your expertise in email content analysis to distinguish
                important emails from the rest, pay attention to the sender and
    avoid invalid emails.

                Make sure to filter for the messages actually directed at the user
    and avoid notifications.

                EMAILS
                -------
                {emails}

                Your final answer MUST be the relevant thread_ids and the sender,
    use bullet points.
                """),
            agent=agent
        )
```

Figure 4.6: Task definition in CrewAI (transcribed from [1], Fig. 4)

Chapter 5

Results and Performance

Do not expect tables of numbers here. The base paper reports worked cases rather than experiments: it builds two multi-agent apps with LangGraph+CrewAI, shows that they run, and says what the pairing made better. I take the two cases in turn. After that I pull out the main findings, and I end the chapter by asking how much these results actually prove.

5.1 Application Case 1: Code Generation with Code Review

The first case writes code and then reviews it (Fig. 5.1). A question goes into a *code generation* node, which writes the code. A *code check* node tests it. If the test fails, the flow goes back to generation, this time with the error attached. If it passes, the code is the answer. Each attempt's status and feedback travel through the shared state as they happen, and the authors say this made code generation more efficient [1]. Reflexion and ReAct argue for exactly this write, test and fix cycle. The difference is that here the loop is a real edge in the graph that you can see and inspect, not something left to the model's judgement.

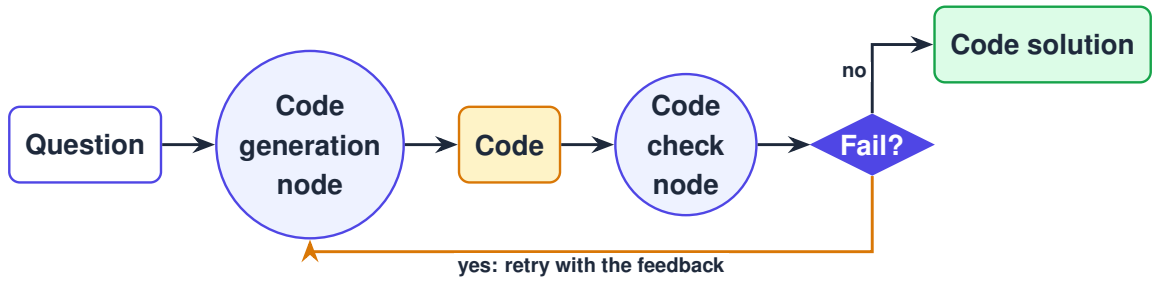


Figure 5.1: Code generation and code check loop built with LangGraph+CrewAI (redrawn from [1], Fig. 5)

5.2 Application Case 2: Automatic Work-Order Processing

Case two is about work orders. These are the tickets an IT or maintenance desk sees all day (Fig. 5.2). First an Ollama embedding model turns the ticket text into a vector. The vector goes into FAISS, which makes it quick to pull up old tickets that look similar. Then a few CrewAI agents, one per task, read the ticket closely and pass their notes to each other. The authors say the agents understood tickets better this way, and got through them faster [1].

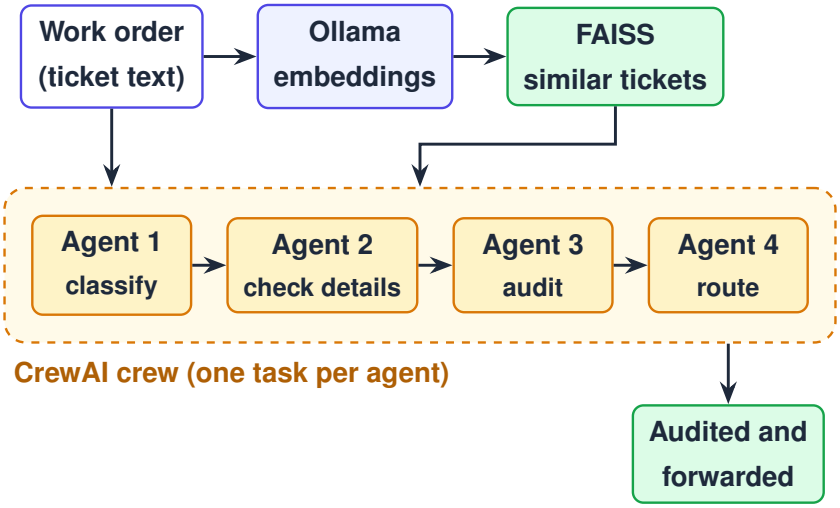


Figure 5.2: Work-order processing with a crew of agents and a vector store (redrawn from the description in [1]; the agents' labels are illustrative)

5.3 Key Findings

The authors draw four conclusions from these cases:

- **Information flows better.** The graph structure makes it easy to pass results between agents.
- **The flow is under control.** Building agents on LangGraph gives the developer a firm grip on the multi-agent workflow.
- **Teamwork improves.** CrewAI's task allocation and resource management make the agents work together better.
- **Harder tasks become possible.** By splitting the work, the agents manage tasks a single agent finds too hard.

5.4 Comparison with Other Approaches

So how does this stack up against the work in Chapter 2? Table 5.1 lines the approaches up against what you would care about if you needed a multi-agent system you can keep under control. Treat it as my own reading of the papers. None of the rows comes from a measurement.

Table 5.1: Qualitative comparison of LLM application approaches

Approach	**Flow set by**	**Loops**	**Saved state**	**Human-in-the-loop**	**Role-based teams**	**Predictable**
Simple chain	Developer	No	No	No	No	High
ReAct agent	LLM	Yes	No	Limited	No	Low
AutoGen	Conversation	Yes	Partial	Yes	Yes	Low
MetaGPT	Fixed SOP	Limited	Documents	No	Yes	Medium
LangGraph alone	Developer (graph)	Yes	Yes (checkpoints)	Yes	Manual	High
LangGraph + CrewAI	**Developer (graph)**	**Yes**	**Yes**	**Yes**	**Yes (crews)**	**High**

5.5 Discussion

I think the paper's real strength is that it makes the fit between LangGraph and CrewAI easy to see. The graph decides what happens next. The crew decides who does it. The three cases help, because they are ordinary jobs: sorting email, checking code, handling tickets. Most offices have all three.

But the results need careful reading. The paper has no numbers at all. There are no success rates, no response times, no costs, and no comparison with a single agent or with another framework. So when the authors say efficiency "improved", they mean that is what they saw in their cases. No test was run to show it. I would like to see the pairing run on a benchmark, measuring task success, model calls and cost against a single agent and against LangGraph alone. Even without that, the paper is a clear, readable guide to building multi-agent apps where control and teamwork live in separate layers.

Chapter 6

Advantages

- **Explicit control.** The developer decides which paths exist. The model can fill in a step but cannot invent a new route, so behaviour is easier to predict and to audit.
- **Shared state.** Every step sees the same, current data. Nothing has to be copied between agents or crammed into a prompt.
- **Loops and retries.** A step can repeat until something is true, say until the tests pass. A plain chain cannot do that.
- **Persistence.** The state is saved after each step, so a run can stop, resume and recover from a crash without starting over [2].
- **Human-in-the-loop.** The graph can pause at any node and wait for a person. That matters before something like sending an email.
- **Easier debugging.** Each node is an ordinary function you can test alone, and LangSmith [12] shows which path a run took.
- **Not tied to one model.** LangGraph and CrewAI both work with several providers, including local models through Ollama.
- **Clear teamwork.** With CrewAI, roles and tasks give every agent a defined job, and existing LangChain agents can join a crew with little extra work.

Chapter 7

Disadvantages

- **Learning curve.** Graphs, typed state, reducers and checkpointers take time to learn. Using two frameworks means learning two sets of ideas.
- **More code for small jobs.** Something a chain does in five lines needs noticeably more setup as a graph.
- **Big graphs get messy.** Once there are many nodes and conditional routes, the flow is hard to picture.
- **Cost and delay.** Loops and several agents mean more model calls. That costs money and makes replies slower.
- **Still depends on the model.** The graph limits the routes, but a bad model output can still choose the wrong one, or do a poor job inside a node.
- **Fast-changing APIs.** Both frameworks change between versions, so code and tutorials go stale quickly.
- **No numbers in the base paper.** The benefits are shown only through cases, so they are not yet backed by measurements.

Chapter 8

Applications

- **Customer support.** A router node works out what a query is about and sends it to a billing agent, a technical agent or a person. The conversation stays in the shared state.
- **RAG question answering.** The graph fetches documents and scores how relevant they are. If they are poor, it rewrites the query and searches again before answering.
- **Code generation and bug fixing.** Write, test and fix in a loop, as in the base paper's code-review case and the authors' later work on automated bug fixing with LangGraph [14].
- **Machine translation.** Text is routed to an agent for the right language [10].
- **Data analysis.** Separate agents build and run machine learning pipelines on Apache Spark [15].
- **Multi-agent teams.** Crews for research, writing and review, one role per agent, as the base paper proposes [1].
- **Ticket automation.** Checking, sorting and forwarding work orders, with similar past cases pulled from a vector database.

Chapter 9

Conclusion

LangGraph treats an LLM app as a graph whose steps all share one state. Branching, looping, memory, a point where a person can step in: a chain has none of these, and a graph has all of them. Nor does the developer have to gamble on a fully autonomous agent and hope it behaves. You decide the routes; the model does the thinking along them.

Duan and Wang show how far this goes when you add CrewAI. LangGraph manages the flow and the state. CrewAI organises agents into teams with roles. Between them they handle jobs that one agent struggles with. Their email assistant, code-review loop and ticket system make the idea concrete, and none of them is exotic.

The weak point is the evaluation. Everything rests on cases, not measurements, so the claimed efficiency gains still need proper benchmarks of success rate, cost and response time. It would also be worth testing how these systems behave as the number of agents grows, and how to keep large graphs readable.

More and more teams now build agentic AI this way, as graphs, when they need it to be trusted. After working on this seminar I understand why. Once the structure is written down, a model's mistakes stop being a mystery. You can point to the node where one happened. You can stop it spreading. Then you fix it.

Appendix A

Bibliography

[1] Z. Duan and J. Wang, “Exploration of LLM multi-agent application implementation based on LangGraph+CrewAI,” *arXiv preprint arXiv:2411.18241*, 2024.

[2] LangChain Inc., “LangGraph documentation,” https://langchain-ai.github.io/langgraph/, 2025.

[3] G. Malewicz, M. H. Austern, A. J. C. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski, “Pregel: A system for large-scale graph processing,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2010, pp. 135–146.

[4] CrewAI Inc., “CrewAI documentation,” https://docs.crewai.com, 2025.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629.

[6] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2303.11366.

[7] Q. Wu, G. Bansal, J. Zhang, Y. Wu *et al.*, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.

[8] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng *et al.*, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2308.00352.

[9] L. Wang, C. Ma, X. Feng, Z. Zhang *et al.*, “A survey on large language model based autonomous agents,” *arXiv preprint arXiv:2308.11432*, 2023.

[10] J. Wang and Z. Duan, “Agent AI with LangGraph: A modular framework for enhancing machine translation using large language models,” *arXiv preprint arXiv:2412.03801*, 2024.

[11] Ollama, “Ollama documentation,” https://github.com/ollama/ollama, 2025.

[12] LangChain Inc., “LangSmith documentation,” https://docs.smith.langchain.com, 2025.

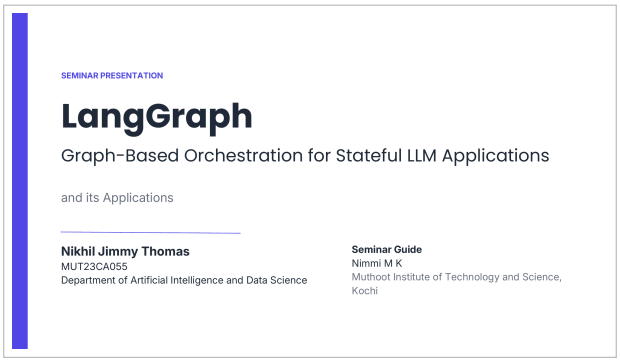
[13] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[14] J. Wang and Z. Duan, “Empirical research on utilizing LLM-based agents for automated bug fixing via LangGraph,” *arXiv preprint arXiv:2502.18465*, 2025.

[15] ——, “Intelligent spark agents: A modular LangGraph framework for scalable, visualized, and enhanced big data machine learning workflows,” *arXiv preprint arXiv:2412.01490*, 2024.

Appendix B

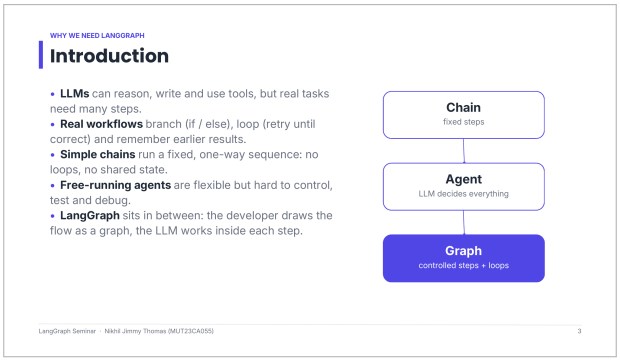
Presentation Slides



Slide 1

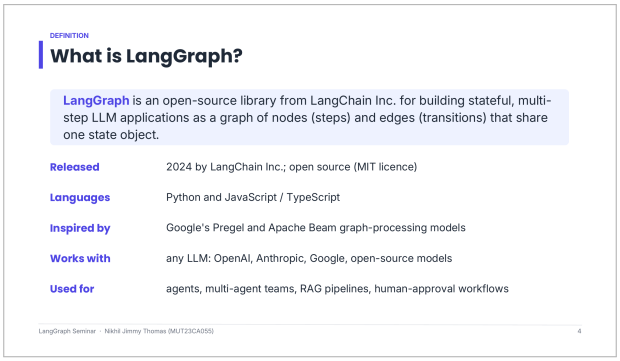


Slide 2

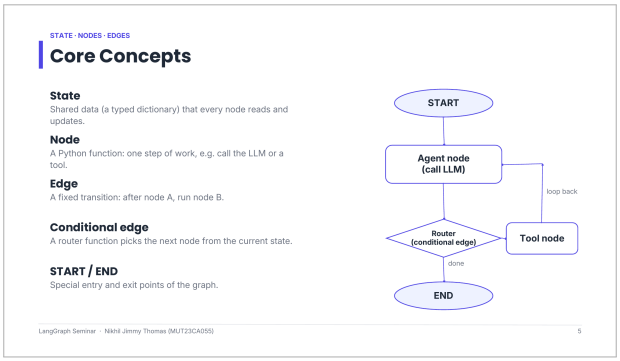


Slide 3

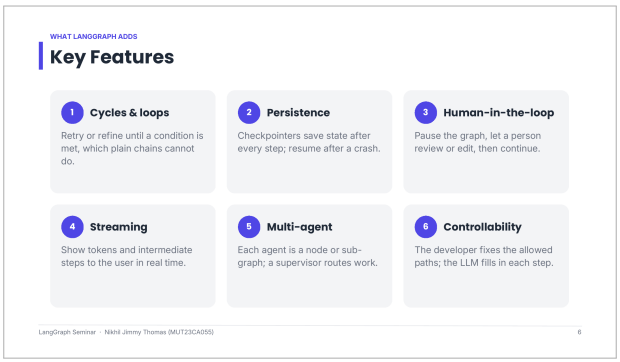
Figure B.1: Seminar presentation slides (1–3)



Slide 4

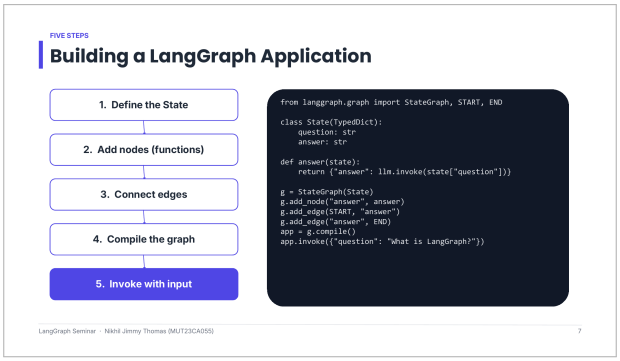


Slide 5



Slide 6

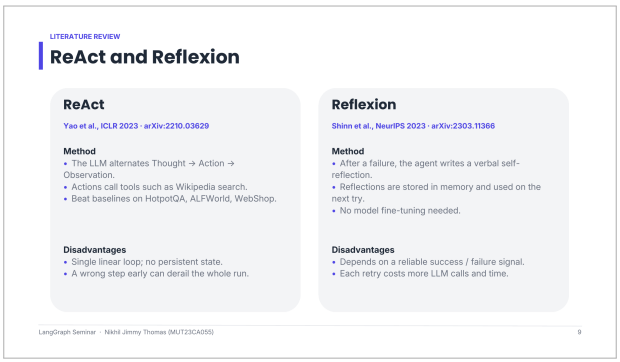
Figure B.2: Seminar presentation slides (4–6)



Slide 7

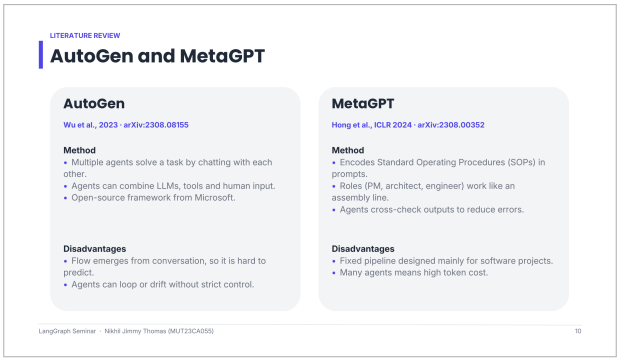


Slide 8

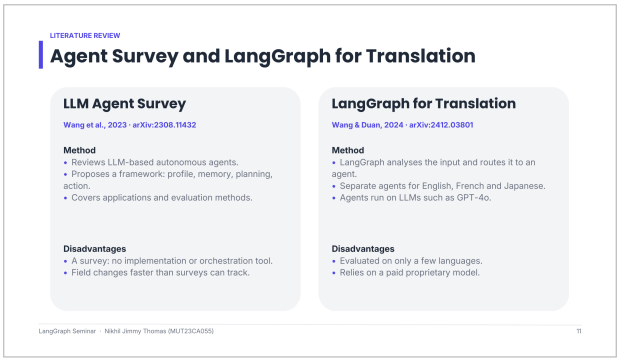


Slide 9

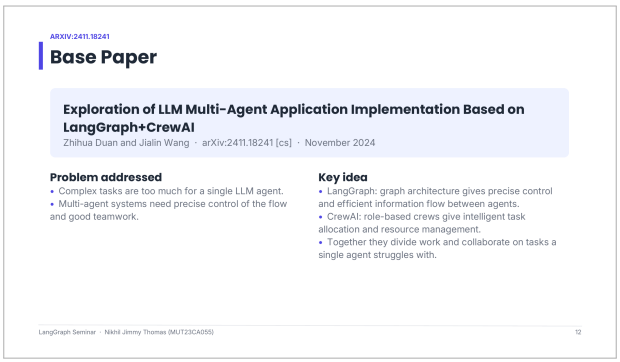
Figure B.3: Seminar presentation slides (7–9)



Slide 10

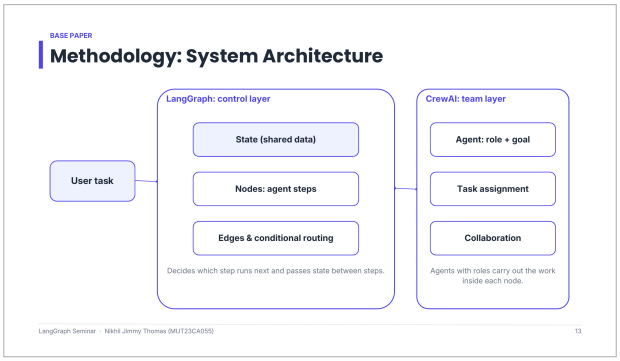


Slide 11

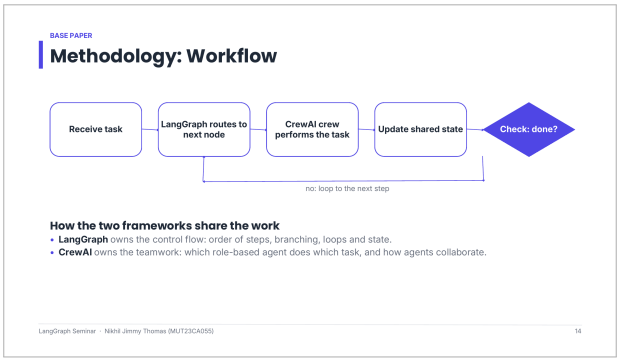


Slide 12

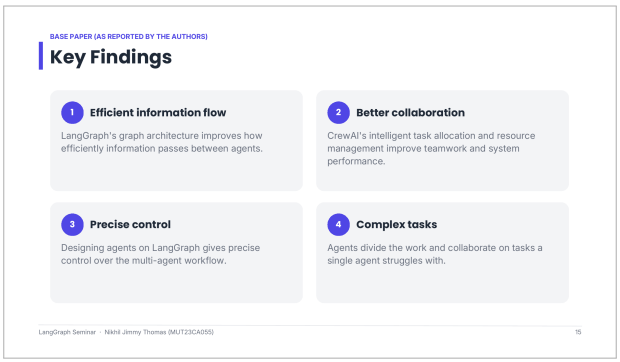
Figure B.4: Seminar presentation slides (10–12)



Slide 13

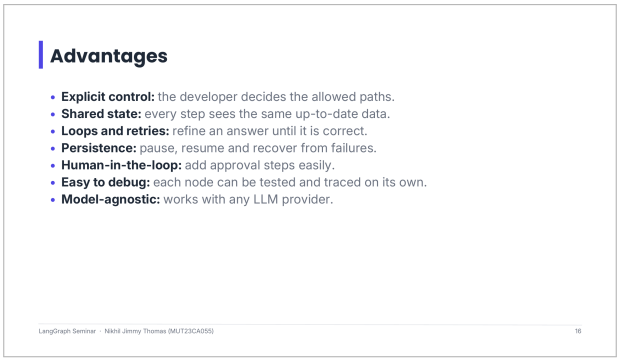


Slide 14

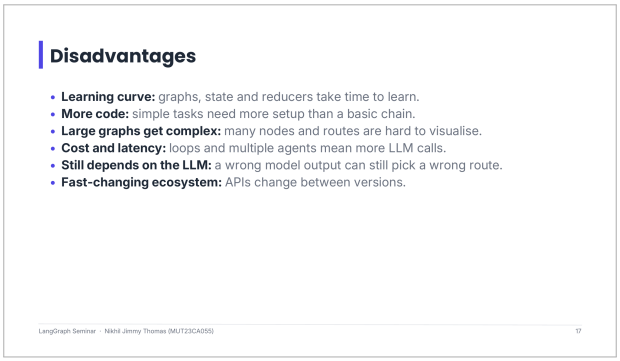


Slide 15

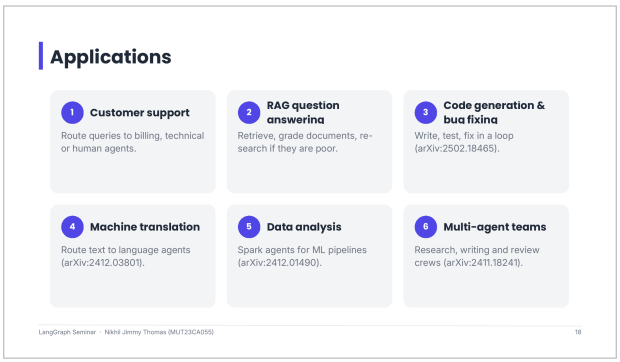
Figure B.5: Seminar presentation slides (13–15)



Slide 16

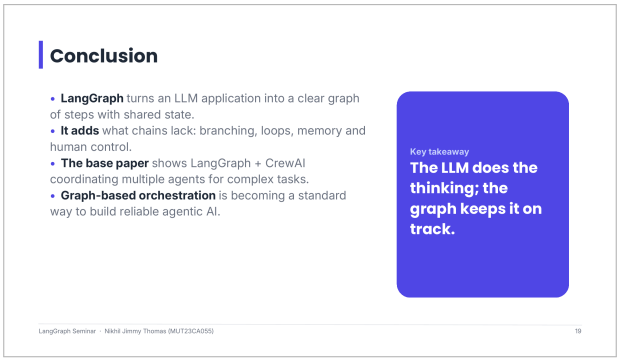


Slide 17

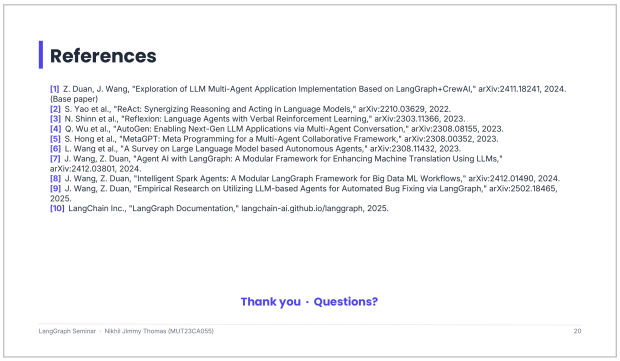


Slide 18

Figure B.6: Seminar presentation slides (16–18)



Slide 19

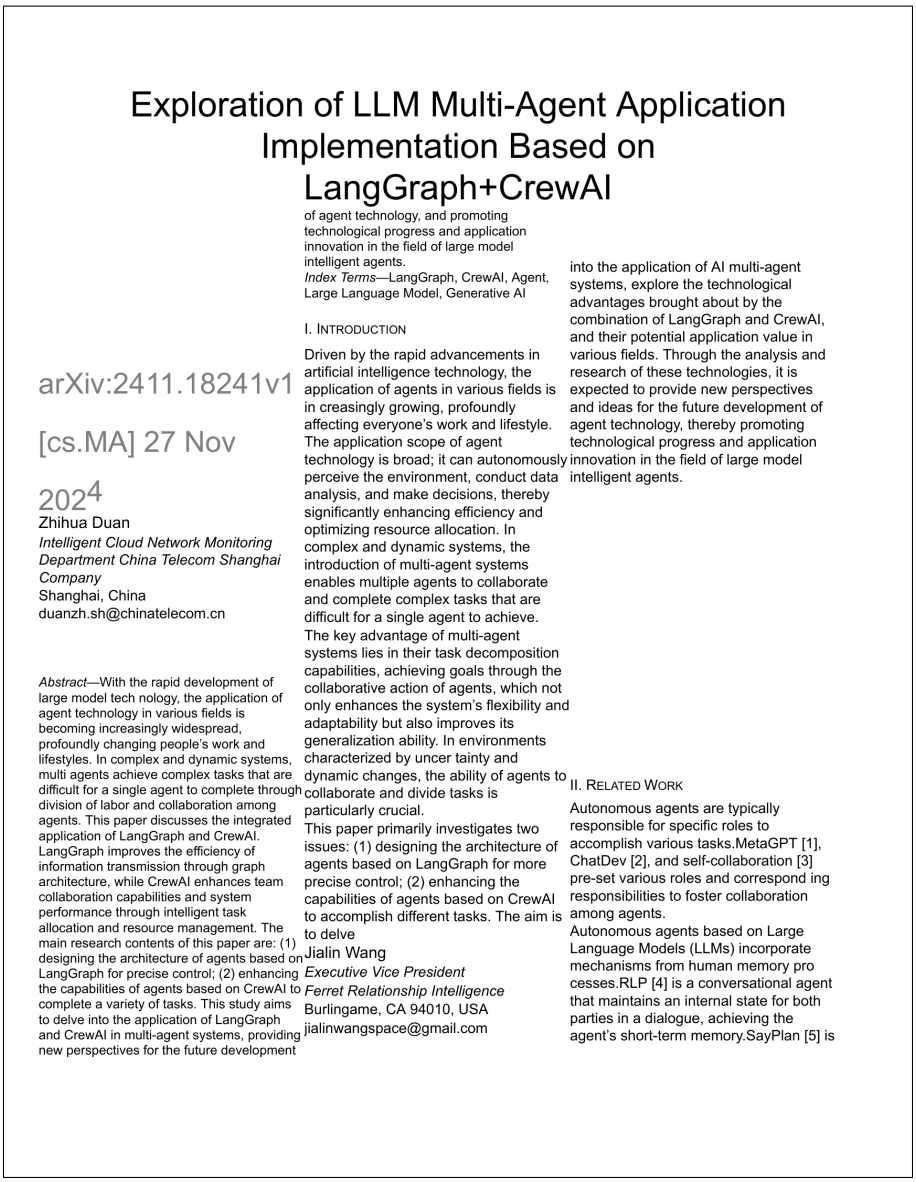


Slide 20

Figure B.7: Seminar presentation slides (19–20)

Appendix C

Sample Reference Papers



Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI

arXiv:2411.18241v1 [cs.MA] 27 Nov 2024

Zhihua Duan
Intelligent Cloud Network Monitoring Department China Telecom Shanghai Company
Shanghai, China
duanzh.sh@chinatelecom.cn

Abstract—With the rapid development of large model tech nology, the application of agent technology in various fields is becoming increasingly widespread, profoundly changing people's work and lifestyles. In complex and dynamic systems, multi agents achieve complex tasks that are difficult for a single agent to complete through division of labor and collaboration among agents. This paper discusses the integrated application of LangGraph and CrewAI. LangGraph improves the efficiency of information transmission through graph architecture, while CrewAI enhances team collaboration capabilities and system performance through intelligent task allocation and resource management. The main research contents of this paper are: (1) designing the architecture of agents based on LangGraph for precise control; (2) enhancing the capabilities of agents based on CrewAI to complete a variety of tasks. This study aims to delve into the application of LangGraph and CrewAI in multi-agent systems, providing new perspectives for the future development of agent technology, and promoting technological progress and application innovation in the field of large model intelligent agents.

Index Terms—LangGraph, CrewAI, Agent, Large Language Model, Generative AI

I. Introduction

Driven by the rapid advancements in artificial intelligence technology, the application of agents in various fields is in creasingly growing, profoundly affecting everyone's work and lifestyle. The application scope of agent technology is broad; it can autonomously perceive the environment, conduct data analysis, and make decisions, thereby significantly enhancing efficiency and optimizing resource allocation. In complex and dynamic systems, the introduction of multi-agent systems enables multiple agents to collaborate and complete complex tasks that are difficult for a single agent to achieve. The key advantage of multi-agent systems lies in their task decomposition capabilities, achieving goals through the collaborative action of agents, which not only enhances the system's flexibility and adaptability but also improves its generalization ability. In environments characterized by uncer tainty and dynamic changes, the ability of agents to collaborate and divide tasks is particularly crucial.

This paper primarily investigates two issues: (1) designing the architecture of agents based on LangGraph for more precise control; (2) enhancing the capabilities of agents based on CrewAI to accomplish different tasks. The aim is to delve into the application of AI multi-agent systems, explore the technological advantages brought about by the combination of LangGraph and CrewAI, and their potential application value in various fields. Through the analysis and research of these technologies, it is expected to provide new perspectives and ideas for the future development of agent technology, thereby promoting technological progress and application innovation in the field of large model intelligent agents.

Jialin Wang
Executive Vice President
Ferret Relationship Intelligence
Burlingame, CA 94010, USA
jialinwangspace@gmail.com

II. Related Work

Autonomous agents are typically responsible for specific roles to accomplish various tasks.MetaGPT [1], ChatDev [2], and self-collaboration [3] pre-set various roles and correspond ing responsibilities to foster collaboration among agents.

Autonomous agents based on Large Language Models (LLMs) incorporate mechanisms from human memory pro cesses.RLP [4] is a conversational agent that maintains an internal state for both parties in a dialogue, achieving the agent's short-term memory.SayPlan [5] is

Figure C.1: Base paper: "Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI" (first page) [1]

Appendix D

Viva Questions and Answers

Common Viva Questions

1. **What is LangGraph?**
   Answer: An open-source library from LangChain for building LLM apps as a graph. Nodes are steps, edges are the moves between them, and all nodes share one state.

2. **How is it different from a LangChain chain?**
   Answer: A chain runs its steps once, top to bottom. A LangGraph graph can branch, loop back, keep shared state and save checkpoints, so it can retry, pause and resume.

3. **What are state, nodes and edges?**
   Answer: The state is the shared data every node reads and updates. A node is a function that does one step. An edge is a move between nodes, and a conditional edge uses a small router to pick the next node from the state.

4. **What does a checkpointer do?**
   Answer: It saves the state after every step. So a run can resume after a crash, be inspected while debugging, or wait for a person's approval.

5. **What does CrewAI add in the base paper?**
   Answer: Teamwork. Agents get roles, goals and tools, and tasks are shared out among them. LangGraph controls the order and the state; a CrewAI crew does the work inside a node.

6. **Explain the email example.**
   Answer: The graph starts at `check_new_emails`. With new mail it goes to `draft_responses`, which runs a CrewAI crew; otherwise it goes to `wait_next_run`. After waiting it loops back and checks again.

7. **Which other tools did the authors use?**

Answer: LangSmith to track runs, an Ollama embedding model to turn ticket text into vectors, and FAISS to store those vectors and find similar ones.

8. **How does LangGraph compare with AutoGen?**
   Answer: In AutoGen the order of work comes out of the agents' conversation, which is flexible but unpredictable. In LangGraph the developer lays out the paths, so it is easier to control and debug.

9. **What is the main weakness of the base paper?**
   Answer: It has no measurements. There are no accuracy, cost or speed figures and no baseline, so the efficiency claims are not yet backed by experiments.

10. **Where is LangGraph used?**
    Answer: Customer-support routing, RAG question answering, code generation and bug fixing, translation, data-analysis pipelines, research and writing teams, and ticket automation.

Appendix E

Similarity Index Report

[Insert Similarity Index Report Here]

Appendix F

Alignment with Sustainable Development Goals (SDG)

Table F.1: Mapping of the seminar topic to UN Sustainable Development Goals

SDG	**Rationale for alignment**
SDG 9: Industry, Innovation and Infrastructure (primary)	Open-source orchestration frameworks such as LangGraph make it easier and cheaper to build reliable AI systems, supporting innovation in industry and research.
SDG 4: Quality Education	Graph-based agents can power tutoring and question-answering assistants, and LangGraph's explicit structure makes agentic AI easier for students to learn.